Parse, Don't Validate — In a Language That Doesn't Want You To

Christian Ekrem · April 7, 2026 · 12-minute read · Typescript, Type-Safety, Functional Programming, Parsing, Domain-Modeling
Source: cekrem.github.io/posts/parse-dont-validate-typescript

Update: If you liked this post, the follow-up — *Effect Without Effect-TS: Algebraic Thinking in Plain TypeScript* — picks up where we left off and takes the ideas further.

I've been thinking about Alexis King's Parse, don't validate again. I do this quite regularly, actually, usually after staring at a TypeScript codebase that's been quietly accumulating `if (user.email)` checks like barnacles. The post is from 2019, and the advice (or rather principle) is way older than that. And yet most TypeScript I read — including, embarrassingly, plenty I've written — still validates instead of parsing.

The pitch, if you haven't read it (you should): a validator says "this thing is fine, please continue." A parser says "give me a blob, and I'll either give you back a more precise type or tell you why I can't." The difference sounds academic, but it isn't: validators throw away what they learned the moment they finish running, while parsers *preserve* it by encoding it in the type. Once you've parsed a string into an `EmailAddress`, the rest of your program never has to wonder again. Peace of mind and more mental capacity for the fun stuff.

In Haskell or Elm or F# this is just how you write code. The language pulls you toward it. In TypeScript… it doesn't. TypeScript will happily let you do the right thing, but it won't insist, and it won't even gently nudge. If anything, structural typing actively undermines the whole game.

Let me show you what I mean.

The validator we've all written

Here's the kind of code I see (and write) constantly:

```
interface User {
  id: number;
  email: string;
  age: number;
}

// The actual validation is naîve and simplistic, but you get the point:
function isValidUser(user: User): boolean {
  if (!user.email.includes("@")) return false;
  if (user.age < 0 || user.age > 150) return false;
  return true;
}

function sendWelcome(user: User) {
  if (!isValidUser(user)) {
    throw new Error("invalid user");
  }
  // ...later, deeper in the call stack:
  emailService.send(user.email, `Welcome, age ${user.age}`);
}
```

Spot the lie? `User.email` is just `string`. `User.age` is just `number`. The validation happened — congrats — but the type system forgot about it the instant `isValidUser` returned. Three function calls deeper, when somebody touches `user.email`, there is *nothing* stopping them from passing it to a function that expects a real email. Because as far as TypeScript is concerned, it's just a string. Same as `""`, same as `"hello"`, same as `"definitely not an email"`.

So what do we do? We validate again somewhere else, add another `if`, write a unit test, and hope for the best. (King has a much better word for this in the original post: "shotgun parsing" — validation scattered everywhere, none of it remembered.)

What we actually want

We want this:

```
function sendWelcome(user: ValidUser) {
  emailService.send(user.email, `Welcome, age ${user.age}`);
}
```

And we want it to be *impossible* to call `sendWelcome` with anything that hasn't been through the parser. No re-checking or "defensive programming"; a `ValidUser` can only have come from one place, and the compiler knows it.

In Elm I'd reach for an opaque type and a smart constructor and be done in about four lines. In TypeScript it's, well, *possible* at least. Just less pleasant.

Branded types, or: lying to the structural type system on purpose

TypeScript is structurally typed, which means two types with the same shape are the same type. `string` is `string` is `string`. There's no `newtype`. There's no `type EmailAddress = String` that produces a genuinely distinct type the way, say, Haskell does it.

The workaround the community has settled on is *branding* — also called *tagging*, also called *nominal typing via intersection*. The cheap version is a string-literal phantom (`{ readonly __brand: "Email" }`) and you'll see it everywhere; the slightly less cheap version uses a `unique symbol` that you don't export from the module, so nobody outside can even *spell* the brand to forge it:

```
declare const EmailBrand: unique symbol;
declare const AgeBrand: unique symbol;

type Email = string & { readonly [EmailBrand]: true };
type Age = number & { readonly [AgeBrand]: true };
```

There is no brand field at runtime. It's a "phantom" — a type-level marker that makes `Email` and `string` incompatible at compile time. The only way to get an `Email` is through a function that knows how, because nothing outside this module can even name the symbol to fake one. (TS5 also lets you flirt with template literal types — `type Email = `${string}@${string}`` — which is fun for a demo and not enough on its own.) That's what lets you make illegal states unrepresentable without leaving the language.

The brand is one-way, by the way: an `Email` is still assignable to `string`, so you get nominal checking on the way into the domain and ordinary structural behavior on the way out. Which is pretty much exactly what you want.

That function is your parser:

```
type ParseError = { kind: "ParseError"; message: string };
type Parsed<T> = { kind: "ok"; value: T } | { kind: "err"; error: ParseError };

function parseEmail(raw: string): Parsed<Email> {
  if (!raw.includes("@")) {
    return { kind: "err", error: { kind: "ParseError", message: "missing @" } };
  }
  // we've checked, now we lie to the type system on purpose
  return { kind: "ok", value: raw as Email };
}

function parseAge(raw: unknown): Parsed<Age> {
  if (
    typeof raw !== "number" ||
    !Number.isInteger(raw) ||
    raw < 0 ||
    raw > 150
  ) {
    return { kind: "err", error: { kind: "ParseError", message: "bad age" } };
  }
  return { kind: "ok", value: raw as Age };
}
```

(The `parseEmail` predicate is embarrassingly thin — a real one would trim, lowercase, and at least pretend to validate the domain part. I'm not, however, writing an email parser in a blog post(!).) The `as Email` hurts a little, and it should. It's the one place where we're allowed to break the rules — the parser is the trusted boundary. Everywhere else in the codebase, you cannot conjure an `Email` out of a `string`. You have to call `parseEmail` and handle both branches. (I'm using `kind: "ok" | "err"` instead of a `success: boolean` discriminant on purpose. A boolean can only ever hold two cases, so the day you need a third one — `"partial"`, say — you're remodeling every consumer. A string discriminant just grows another case, and the exhaustiveness check further down will point at every switch you forgot to update.)

Compare this to the throw-and-pray validator we started with: its failure mode is an exception, which is invisible to the type system. The parser's signature tells you everything that can happen; nothing extra is hiding in the call stack.

Now the domain type. I want to name two things that usually get conflated: the raw blob that came off the wire, and the thing I've earned the right to trust.

```
declare const UserIdBrand: unique symbol;
type UserId = number & { readonly [UserIdBrand]: true };

type UnvalidatedUser = {
  id: unknown;
  email: unknown;
  age: unknown;
};

type ValidUser = {
  readonly id: UserId;
  readonly email: Email;
  readonly age: Age;
};

function parseUserId(raw: unknown): Parsed<UserId> {
  if (typeof raw !== "number" || !Number.isInteger(raw) || raw < 0) {
    return { kind: "err", error: { kind: "ParseError", message: "bad id" } };
  }
  return { kind: "ok", value: raw as UserId };
}

function parseUser(raw: unknown): Parsed<ValidUser> {
  if (typeof raw !== "object" || raw === null) {
    return {
      kind: "err",
      error: { kind: "ParseError", message: "not an object" },
    };
  }
  if (!("id" in raw) || !("email" in raw) || !("age" in raw)) {
    return {
      kind: "err",
      error: { kind: "ParseError", message: "missing fields" },
    };
  }
  if (typeof raw.email !== "string") {
    return {
      kind: "err",
      error: { kind: "ParseError", message: "email not a string" },
    };
  }

  const id = parseUserId(raw.id);
  if (id.kind === "err") return id;

  const email = parseEmail(raw.email);
  if (email.kind === "err") return email;

  const age = parseAge(raw.age);
  if (age.kind === "err") return age;

  return {
    kind: "ok",
    value: { id: id.value, email: email.value, age: age.value },
```

```
  };
}
```

Naming `UnvalidatedUser` separately from `ValidUser` is a small DDD move that pays for itself: stuff goes in raw, stuff comes out trusted, and the boundary is a function. `id` is also branded, because a `UserId` that can't be passed where an `OrderId` is expected is one of the cheapest wins in the whole technique. (No more `as Record<string, unknown>` either; if I'm writing a post about not lying to the type system, I shouldn't lie to the type system.)

This is uglier than the F# or Elm equivalent, by far. I won't pretend otherwise. The early-return-on-error pattern is the closest thing TypeScript has to a `Result` monad without dragging in a library, and it gets repetitive. (You *can* use Effect or neverthrow or fp-ts to clean this up, and for anything bigger than a toy I would. But I want to show what the language gives you out of the box, because the principle survives even when the syntax doesn't.)

The payoff is that `sendWelcome(user: ValidUser)` is now genuinely safe. There is no path through your codebase that produces a `ValidUser` without going through `parseUser`. The type *is* the proof: the validation didn't get thrown away.

Where TypeScript fights you

A few things still grate.

The first is that `as Email` cast inside `parseEmail`. In a real nominal language, the smart constructor doesn't have to lie — it returns the new type because the new type is genuinely different. In TypeScript, the brand is fictional, so you have to assert your way past it. The discipline this requires is: *only the parser is allowed to do that assertion*. If the cast leaks anywhere else in the codebase, the whole scheme collapses. I've taken to putting parsers in their own module and treating any `as Brand<...>` outside that module as a bug. (A custom ESLint rule helps.)

The second is exhaustiveness. Discriminated unions are TypeScript's killer feature for this style — they're as close as the language gets to Elm's custom types — and the language *does* do exhaustiveness checking via `never`-narrowing; what it lacks is a dedicated `match` expression, so you have to write the `never` trick by hand and remember to write it:

```
function describe(result: Parsed<ValidUser>): string {
  switch (result.kind) {
    case "ok":
      return `user ${result.value.id}`;
    case "err":
      return `failed: ${result.error.message}`;
    default: {
      const _exhaustive: never = result;
      return _exhaustive;
    }
  }
}
```

Add a third variant to `Parsed` and the `never` assignment fails and the compiler tells you exactly where to look. Compare to Elm, where forgetting a branch is a compile error you literally cannot ignore.

(And while we're here: `satisfies` is the other modern escape hatch worth knowing — `const x = { ... } satisfies Config` checks against the type without widening, so you keep the precise literal type and still get the safety. It's the polite version of the cast.)

The third thing that grates is `JSON.parse`. It returns `any`, which is the worst type in the language and the entire reason this post exists. Annotate it as `unknown` immediately — `const raw: unknown = JSON.parse(input)` — and let the parser take it from there. `JSON.parse` isn't a validator's evil cousin; it's a deserializer. It turns bytes into a JS value. Whether that value is a `User` is a completely separate question, and it's the one your parser exists to answer.

What about Zod?

Zod is great. So is io-ts. So is valibot. Use them. They're the ergonomic version of everything I just wrote — a schema-first DSL that gives you a parser and a TypeScript type from the same definition:

```
import { z } from "zod";

const ValidUserSchema = z.object({
  id: z.number().int(),
  email: z.string().email().brand<"Email">(),
  age: z.number().int().min(0).max(150).brand<"Age">(),
});

type ValidUser = z.infer<typeof ValidUserSchema>;

const result = ValidUserSchema.safeParse(rawInput);
```

`safeParse` returns `{ success: true, data }` or `{ success: false, error }` — same shape as what I built above, different field names. The `.brand()` call is purely type-level, exactly like the hand-rolled symbol trick; nothing happens at runtime. What you get is the parser and the

type from one definition, which structurally enforces the parser/type co-location boundary I was asking you to enforce by hand a few sections ago. That alone is worth the dependency.

But — and this is the part I keep coming back to — Zod doesn't change the *mindset problem*. It just makes the right thing easier. You still have to choose to use it at every boundary. You still have to resist the temptation to type-assert your way out of an error message. You still have to remember that a `User` from the network is not a `User` until something has parsed it. Zod makes that discipline a lot cheaper. It doesn't make it optional.

(I mentioned this briefly in *Why TypeScript Won't Save You*, and it's the same point: the language won't enforce the boundary, so you have to.)

The smaller principle

If I had to compress King's idea into a sentence I'd actually remember at 11pm before a release: *make the type system carry the proof, not your memory*. Every time you check something and don't encode the result in a type, you're asking your future self to remember. Future you will not remember. Future you is debugging a different bug, on three hours of sleep, and is going to assume the validation already happened because of course it did, look at all these `if` statements.

In TypeScript this means leaning on three things the language *does* give you, even if it gives them grudgingly: branded types for nominal-ish identity, discriminated unions for honest error handling, and a strict boundary between `unknown` (what came from outside) and your domain types (what you've earned the right to trust). It's nowhere near as clean as Elm, but it beats the alternative by a good margin.

I still write validators sometimes. I'm not going to pretend I refactor every codebase I touch into a parsing pipeline — that would be a lie, and also probably bad use of my time. But when I find myself adding the third defensive `if` in three different files, all checking the same thing, I know what's happened. I validated when I should have parsed. The information is there. It just isn't in the type.

That's usually when I go back and read King's post one more time.

Christian Ekrem · cekrem.github.io · Originally published April 7, 2026